Security Findings Summary

Security Findings Summary

Project: princesspink90 (VIP venue booking platform) **Report date:** 2026-07-15
Scanner: Lovable Supabase security scan + scripts/security-regression-scan.mjs **Scope:** 8 findings triaged in this review cycle.

Executive summary

#	Internal ID	Class	Status
1	nowpayments_redirect	Open redirect	**Fixed**
2	billing_portal_open_redirect	Open redirect	**Fixed** (code removed)
3	events_public_select_star_leak	Data over-exposure	**Fixed**
4	analytics_events_open_insert	Insert abuse	**Mitigated / ignored**
5	memberships_user_update_perks	Privilege escalation	**Mitigated / ignored**
6	rsvps_user_update_policy	Privilege escalation	**Mitigated / ignored**
7	SUPA_anon_security_definer_function_executable	Advisory	**Accepted / ignored**
8	SUPA_authenticated_security_definer_function_executable	Advisory	**Accepted / ignored**

Regression control: A new offline CI step (bun run lint:security-regression) statically blocks reintroduction of open-redirect, select('*') on guest-exposed tables, and unpinned SECURITY DEFINER migrations. Baseline: security/regression-baseline.json.

Post-fix rescan: All eight IDs no longer appear as findings. Two new advisory-only warnings surfaced (private_room_bookings realtime monitoring reminder, content-media bucket missing UPDATE policy — non-exploitable).

Fixed findings — before / after

1. nowpayments_redirect — Open redirect on crypto checkout

- **Before:** bookingInvoice.functions.ts and nowpayments.functions.ts accepted a client-supplied returnOrigin and passed it straight into success_url, cancel_url, and ipn_callback_url. An

attacker could craft a checkout link that returned users to an attacker-controlled origin after payment.
- **After:** All redirect URLs are now built from `resolveAppOrigin(getRequest())`, which trusts only the server-known origin (env override → `x-forwarded-host` → request host, with an allow-list). `returnOrigin` remains an optional input for backwards compatibility but is no longer used to construct URLs.

2. `billing_portal_open_redirect` — Open redirect on billing portal return URL

- **Before:** `createBillingPortalSession` accepted an unvalidated `return_url` argument.
- **After:** The function was removed from the codebase entirely; a stub test guards against reintroduction. No live surface remains.

3. `events_public_select_star_leak` — Sensitive event columns leaked to guests

- **Before:** `getPublicEventById` used `.select('*')` on `public.events`, exposing `host_id`, `insurance_policy_number`, `permit_details`, `compliance_notes`, and `legal_capacity` to unauthenticated visitors.
- **After:** `getPublicEventById` uses an explicit guest-facing column allow-list (`id, title, tagline, description, venue_name, city, address, starts_at, ends_at, dress_code, theme, cover_image_url, ticket_price_cents, waiver_text, capacity, is_private, published`). Sensitive columns are excluded at the query level, not just at the RLS layer.

Mitigated findings (accepted with controls)

4. `analytics_events_open_insert`

- **Risk:** Anonymous INSERT on `analytics_events` could allow event-log flooding or forged user IDs.
- **Mitigation in place:** Insert policy enforces an event-type allow-list plus `user_id IS NULL OR user_id = auth.uid()`. Anonymous inserts cannot forge another user's identity, and unknown event types are rejected.

5. `memberships_user_update_perks`

- **Risk:** A member could UPDATE their own membership row and grant themselves higher-tier perks, extend `expires_at`, or unrevoke.
- **Mitigation in place:** `memberships_block_user_field_tamper` BEFORE UPDATE trigger locks `kind`, `expires_at`, `amount_cents`, `environment`, `private_session_bundle_*`, `event_ticket_*`, `private_session_*`, `revoked_at`, `suspended_at`, `revocation_reason`, and `user_id` for non-admins. Any tamper attempt raises.

6. `rsvps_user_update_policy`

- **Risk:** An attendee could UPDATE their RSVP row and forge check-in state, waiver acceptance, entry codes, or ticket codes.

- **Mitigation in place:** rsvps_block_user_field_tamper BEFORE UPDATE trigger locks checked_in_at, checked_in_by, door_notes, waiver_signature, waiver_accepted_at, entry_code, entry_phrase, ticket_code, user_id, and event_id for non-admin/non-host actors.

Accepted advisories (linter false positives)

7 & 8. SUPA_anon/authenticated_security_definer_function_executable

- **Advisory:** Supabase's linter flags every SECURITY DEFINER function callable by anon or authenticated.
- **Why accepted:** In this project those functions (has_role, has_age_verification, user_can_access_content, get_private_room_busy, and admin RPCs) intentionally require EXECUTE for anon/authenticated because RLS policies and admin RPCs call them from inside policy expressions. Every admin RPC self-checks has_role(auth.uid(),'admin') with a locked search_path. Documented in security memory.

Regression prevention

- scripts/security-regression-scan.mjs — static scanner (no secrets, no network) enforced in CI (.github/workflows/ci.yml, verify job).
- security/regression-baseline.json — baselines 5 pre-existing hits with per-entry rationale; new hits fail the build.
- Rules enforced:
  1. Client-supplied origin (returnOrigin / originOverride / redirectOrigin / clientOrigin) may not flow into success_url / cancel_url / ipn_callback_url / return_url / redirect_uri without resolveAppOrigin.
  2. .select('*') is forbidden in server code against events, profiles, memberships.
  3. CREATE FUNCTION ... SECURITY DEFINER migrations must pin SET search_path.

New advisory findings from post-fix rescan (informational, no action required)

- **private_room_bookings realtime channel** — currently protected by owner/admin SELECT policies; flagged only as a reminder to review before broadening SELECT policies.
- **content-media storage bucket missing UPDATE policy** — functional gap only; not exploitable. Add a policy scoped to auth.uid() = owner if metadata updates are ever needed.